

[JAVASCRIPT]- STOCKAGE DE DONNÉES DANS LE NAVIGATEUR

Les cookies en JavaScript

Objet

Dans cette séquence, nous allons voir ce que sont les cookies et comment créer, modifier ou supprimer des cookies en JavaScript.

Qu'est-ce qu'un cookie et quel est l'intérêt d'un cookie ?

Un cookie est un petit fichier qui ne contient généralement qu'une seule donnée et qui va être stocké directement dans le navigateur d'un utilisateur.

Le plus souvent, les cookies sont mis en place (créés) côté serveur et vont être envoyés avec une page lorsque l'utilisateur demande à y accéder.

Les cookies sont très pratiques car ils permettent de conserver des informations envoyées par l'utilisateur et donc de pouvoir s'en resservir et cela de manière relativement simple.

Les cookies vont nous permettre d'enregistrer des informations à propos de l'utilisateur comme une liste de préférences indiquées (par exemple : « je préfère que ce site utilise son thème foncé » ou « je ne souhaite plus voir ce message ») ou vont encore notamment pouvoir servir aux utilisateurs à se connecter plus facilement à un site en gardant en mémoire leurs informations de connexion.

Expliquons immédiatement ce qu'il se passe dans ce dernier cas. Pour cela, imaginons que nous possédions un site sur lequel les utilisateurs peuvent s'enregistrer. La première fois qu'un utilisateur cherche à accéder à la page de connexion, le navigateur contacte le serveur qui renvoie la page et renvoie également un cookie qui va être stocké dans le navigateur du visiteur et qui va enregistrer ses informations de connexion.

L'utilisateur s'enregistre puis se déconnecte ensuite du site. Le lendemain, il revient sur notre site. Cette fois-ci, le navigateur va, en plus de demander au serveur d'envoyer la page, envoyer le cookie avec les informations de connexion. Ainsi, le serveur va pouvoir identifier l'utilisateur et le connecter automatiquement au site.

Un cookie est-il dangereux ?

Contrairement aux idées reçues, les cookies ne sont pas dangereux en soi : ce ne sont que des petits fichiers stockant une information.

En revanche, le danger réside dans la gestion des cookies par l'utilisateur. En effet, rappelons que les cookies sont toujours stockés dans le navigateur de nos visiteurs. Nous n'y avons donc jamais directement accès et c'est l'utilisateur qui va décider quels cookies il va accepter et lesquels il va refuser.

L'autre danger des cookies réside dans le cas où un programme malveillant arrive à intercepter des cookies et donc les informations parfois sensibles qu'ils contiennent. Cela peut arriver dans le cas où un utilisateur se fait duper ou dans le cas d'une attaque contre notre site.

Quoiqu'il en soit, aujourd'hui, quasiment tous les sites utilisent des cookies car ces derniers apportent une réelle aisance de navigation pour les visiteurs et permettent à de nombreux programmes de fonctionner plus rapidement.

L'enjeu pour nous va être de sécuriser notre site et de faire attention aux différentes informations demandées et à l'utilisation de ces informations.

Obtenir la liste des cookies et créer un cookie en JavaScript

Bien que la majorité des cookies sont initiés côté serveur, on va également pouvoir créer des cookies côté client grâce au JavaScript. Pour cela, on va utiliser le descripteur d'accesseur `document.cookie`.

Un descripteur d'accesseur est une propriété décrite par une paire d'accesseur/mutateur (getter/setter) qui sont des fonctions.

Le descripteur d'accesseur ou la « propriété accesseur » `document.cookie` possède une paire de fonctions getter et setter natives. Cela signifie simplement qu'on va pouvoir accéder aux cookies et écrire de nouveaux cookies avec `document.cookie` sans impacter les cookies déjà créés.

Pour créer un cookie, à minima il faut lui passer un nom et une valeur comme ceci :

```
Document.cookie= 'user=Pierre-Antoine' ; // Create or Update a cookie named 'user'
```

Pour obtenir la liste des cookies relatifs au domaine, nous allons à nouveau utiliser `document.cookie` sans fournir de valeur comme ceci :

```
document.cookie = 'user=Pierre-Antoine' ; // Create or Update a  
cookie named 'user'  
alert(document.cookie);
```

Note : si vous tentez d'exécuter ce code directement dans votre navigateur sans passer par un serveur (local ou autre), aucun cookie ne sera créé.

Les options des cookies

En plus d'une paire nom=valeur, on va également pouvoir définir des options pour nos cookies comme leur domaine de validité ou encore leur date d'expiration (aucun cookie n'est stocké définitivement dans un navigateur).

La portée des cookies : chemin (répertoire) et domaine d'accessibilité

On va déjà pouvoir préciser un répertoire dans lequel le cookie est accessible avec l'option `path`. Le chemin fourni doit être absolu. Par défaut, un cookie est accessible dans la page courante.

Par exemple, un cookie défini avec `path = /cours` sera disponible dans les pages `/cours` et `/cours/...` mais pas dans les pages `/home` ou `/articles`.

Généralement, on écrira `path = /` pour rendre le cookie accessible à partir de toutes les pages du site Web, c'est-à-dire sur l'ensemble du domaine ou du sous domaine.

```
// Create or Update a cookie named 'user'  
document.cookie = 'user=Pierre-Antoine' ; path = '/' ;
```

L'option `domain` permet de préciser le domaine sur lequel le cookie doit être accessible.

Par défaut, un cookie est disponible dans le domaine ou dans le sous domaine dans lequel il a été créé uniquement mais pas dans les autres sous domaines.

Notez que cette option est limitée à l'ensemble du domaine principal et des sous domaines dans lequel le cookie a été créé et ceci pour des raisons de sécurité évidentes.

Par exemple, si je crée un cookie pour la page `epul3A.local` sans préciser de domaine, le cookie sera disponible dans le domaine `epul3A.local` mais pas dans un sous domaine `cours.epul3A.local` ni sur un autre domaine.

Si je mentionne explicitement `domain=epul3A.local` lors de la création du cookie, en revanche, mon cookie sera disponible sur le domaine et sur l'ensemble des sous domaines liés à `epul3A.local`.

```
// Create or Update a cookie named 'user'
document.cookie = 'user=Pierre-Antoine ; path = / ;
domain=epul3A.local' ;
```

L'âge maximal et la date d'expiration des cookies

Par défaut, un cookie est supprimé dès que le navigateur est fermé. L'option `expires` permet de préciser une date d'expiration pour un cookie, afin de faire en sorte qu'un cookie soit conservé plus longtemps pour pouvoir être réutilisé dans le futur.

Pour que cette option fonctionne correctement, il faudra bien fournir un format de date spécifique et avec le fuseau horaire GMT. On peut utiliser la méthode `toUTCString()` de l'objet `Date` pour s'assurer que notre date possède le bon format.

On va ainsi par exemple pouvoir définir un cookie qui devra expirer (être supprimé) exactement 24h après sa création comme cela :

```
let date = Date(Date.now() + 86400000) ; 86400000ms = 1 day
date = date.toUTCString() ;
// Create or Update a cookie named 'user'
document.cookie = 'user=Pierre-Antoine ; path = / ;
domain=epul3A.local ; expires = ' + date ;
```

Notez qu'on peut également utiliser l'option `max-age` pour définir la date d'expiration d'un cookie en secondes à partir du moment actuel. Cette option est une alternative à `expires` qui nous permet d'utiliser des nombres.

```
// Create or Update a cookie named 'user'
document.cookie = 'user=Pierre-Antoine ; path = / ;
domain=epul3A.local ; max-age = 86400000' ;
```

Les cookies et la sécurité

L'option `secure` permet d'indiquer qu'un cookie doit être envoyé uniquement via HTTPS et ne pas l'être via HTTP. Cette option est très utile si un cookie possède des données sensibles qui ne doivent pas être envoyées sans cryptage.

```
// Create or Update a cookie named 'user'
document.cookie = 'user=Pierre-Antoine ; path = / ;
domain=epul3A.local ; secure' ;
```

L'option `samesite` empêche le navigateur d'envoyer un cookie lors d'une requête cross-site. Cette option offre une bonne protection contre les attaques de type XSRF (cross-site request forgery).

Pour comprendre comment fonctionne ce type d'attaques et à quoi sert l'option `samesite`, considérons l'exemple suivant. Imaginons que vous soyez connecté à un site marchand et que vous possédiez donc un cookie qui sert à vous identifier stocké dans votre navigateur.

Vous ouvrez un second onglet et allez sur un autre site. Ce site est un site malveillant qui possède un formulaire. Ce formulaire est directement envoyé sur le site marchand (en précisant son adresse via l'attribut `action`) et ses champs ont pour but de vous faire acheter quelque chose sur le premier site marchand.

Lorsque vous validez le formulaire, celui-ci est directement envoyé sur le site marchand et votre navigateur envoie également votre cookie d'identification puisque celui-ci est envoyé à chaque fois que vous visitez ce site. Le site marchand vous identifie donc automatiquement et votre achat est effectué sans que vous ne l'ayez voulu. C'est le principe d'une attaque de type cross-site request forgery.

L'option `samesite` permet de se prémunir contre ce type d'attaque. Pour cela, on va pouvoir choisir parmi l'une de ces deux valeurs :

- `samesite="strict"` indique qu'un cookie ne doit jamais être envoyé si l'utilisateur arrive sur le site depuis un autre site ;
- `samesite="lax"` possède les mêmes caractéristiques que la valeur `strict` à la différence que les cookies provenant de requêtes de type `get` de navigation top level (requêtes qui modifient l'URL dans la barre d'adresse du navigateur) seront envoyés.

```
// Create or Update a cookie named 'user'  
document.cookie = 'user=Pierre-Antoine ; path = / ;  
domain=epul3A.local ; samesite = lax' ;
```

Cookies JavaScript et HttpOnly

L'option `httpOnly` ne dépend pas du JavaScript mais va avoir un effet important sur l'utilisation des cookies en JavaScript et nous devons donc la mentionner ici.

Ici, vous devez savoir que le serveur utilise un en-tête (header) `Set-Cookie` pour définir un cookie. En définissant le cookie, il va également pouvoir ajouter une option `httpOnly`.

Cette option interdit tout simplement tout accès au cookie au JavaScript. Nous ne pouvons pas voir ce cookie ni le manipuler avec `document.cookie`.

Cette option est utilisée pour se prémunir d'attaques XSS (cross-site scripting), qui sont des attaques qui reposent sur l'injection de code JavaScript dans une page avec l'intention que l'utilisateur ou que le site lui-même exécute ce code qui va pouvoir récupérer des informations ou créer des dégâts sur le site.

Modifier ou supprimer un cookie en JavaScript

Pour modifier un cookie, il suffit de le réécrire avec le même nom et en changeant les autres informations.

Notez qu'on ne va pas pouvoir changer le nom d'un cookie : si l'on change de nom, cela sera considéré comme un autre cookie et ça n'effacera pas le premier.

Pour supprimer un cookie, la méthode la plus simple est de le réécrire sans valeur et en précisant cette fois-ci une date d'expiration passée.

```
let date = Date(Date.now() + 86400000) ; 86400000ms = 1 day
date = date.toUTCString() ;
// Create or Update a cookie named 'user'
document.cookie = 'user=Pierre-Antoine ; path = / ;
domain=epul3A.local ; expires = ' + date ;

// Deletes the cookie by passing it an expiration date in the past
Document.cookie = 'user = ; path = / ; expires= Thu, 01 Jan 1970
00 :00 :00 UTC' ;
```

L'API Web Storage : `localStorage` et `sessionStorage` en JavaScript

Les cookies permettent de stocker des informations côté client. Cependant, ce n'est pas le seul outil dont nous disposons pour stocker des données dans le navigateur des visiteurs. Nous pouvons également utiliser l'une des deux APIs Web Storage ou IndexedDB.

Présentation de l'API Web Storage et des propriétés `localStorage` et `sessionStorage`

L'API Web Storage permet de stocker des données sous forme de paires clefs/valeurs qui doivent obligatoirement être des chaînes de caractères dans le navigateur de vos visiteurs.

Pour stocker des données avec Web Storage, on va pouvoir utiliser les propriétés (qui sont avant tout des objets) `localStorage` et `sessionStorage`. On va utiliser ces propriétés avec l'objet implicite `Window`.

Pour être tout à fait précis, un objet `Storage` est créé lorsqu'on utilise une de ces propriétés. On va pouvoir manipuler les données à travers cet objet. Notez que l'objet de stockage créé est différent pour `localStorage` et `sessionStorage`.

La principale différence entre `localStorage` et `sessionStorage` est la suivante : dans le cas où on utilise `sessionStorage`, les données enregistrées ne vont subsister qu'après un rechargement de la page courante tandis que si on utilise `localStorage` les données vont subsister même après qu'un visiteur ait quitté son navigateur.

Pour cette raison, la propriété `localStorage` est beaucoup plus utilisée que `sessionStorage`. Nous allons donc particulièrement nous concentrer sur cette première ici. Dans tous les cas, ces deux objets disposent des mêmes méthodes et propriétés, vous n'aurez donc aucun mal à utiliser le second si vous comprenez comment utiliser le premier.

Pourquoi utiliser des objets de stockage plutôt que des cookies ?

Chaque système de stockage va posséder des forces et des champs d'application différents.

Les cookies vont être très utiles pour stocker un petit nombre de données et notamment pour stocker des données d'identification (données de connexion).

Cela est dû au fait que les cookies vont être envoyés au serveur en même temps que chaque requête, ce qui fait que le serveur va pouvoir, en utilisant les données fournies par ceux-ci, identifier immédiatement un visiteur.

D'un autre côté, les autres systèmes de stockage dans le navigateur comme l'API Web Storage stockent des données qui vont rester dans le navigateur : les objets ne vont pas être envoyés au serveur.

Cela fait qu'on va pouvoir stocker un nombre beaucoup plus important de données sans ralentir l'exécution du script comme le ferait un cookie à cause du transfert des données au serveur.

En plus de cela, l'API Web Storage va nous permettre de stocker des données plus simplement que les cookies et applique la politique de même origine, ce qui limite les problèmes de sécurité.

Une origine est la combinaison d'un protocole, un hôte et d'un numéro de port. La politique de même origine indique qu'il n'est pas possible d'accéder à un contenu d'une certaine origine depuis une autre origine.

Les propriétés et méthodes de localStorage et de sessionStorage

Les objets `localStorage` et `sessionStorage` vont nous fournir les propriétés et méthodes suivantes :

- `setItem()` : permet de stocker une paire clef/valeur. Prend une clef et une valeur en arguments ;
- `getItem()` : permet d'obtenir une valeur liée à une clef. Prend une clef en argument ;
- `removeItem()` : permet de supprimer une paire clef/valeur. Prend une clef en argument ;
- `clear()` : permet de supprimer tous les objets de stockage. Ne prend pas d'argument ;
- `key()` : permet d'obtenir une clef située à une certaine position. Prend un index en argument ;
- `length` : permet d'obtenir le nombre de données stockées.

Utiliser l'API Web Storage – Exemple pratique

Dans cet exemple, on va imaginer que l'on gère un site et on veut proposer un thème sombre à nos utilisateurs. Ici, on va se contenter de changer la couleur de fond de la page.

On va enregistrer le choix fait par l'utilisateur en utilisant `localStorage` afin que celui-ci soit conservé pour ses prochaines visites.

Côté HTML, on va utiliser un élément de formulaire pour laisser la possibilité à l'utilisateur de choisir sa couleur de fond.

```
<!doctype html>
<html lang="fr">

  <head>
    <title>[EPUL3A] : Etude JavaScript</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <link href="./css/style.css" rel="stylesheet">
    <script src="./js/myTheme.js"
type="text/javascript"></script>
  </head>

  <body>
    <form>
      <div>
        <label for=bgtheme '> Choisissez un thème (hexa)
:</label>
        <input class='text ' id='bgtheme ' value = 'FAFAFA '
pattern = '[a-fA-F0-9]{6}' '>
      </div>
    </form>
  </body>
</html>
```

En JavaScript, on va déjà cibler l'élément `html` auquel on va ajouter la couleur de fond ainsi que l'élément de formulaire pour récupérer la valeur inscrite par l'utilisateur.

```
let htmlElt = document.querySelector('html');
let bgColor = document.getElementById('bgtheme');
```

Ensuite, dans notre script, on va commencer par vérifier si l'objet de stockage qu'on souhaite créer est déjà présent (cas d'un utilisateur revenant sur notre site) ou pas (cas d'un nouvel utilisateur).

```
if (localStorage.getItem('bgtheme')){
    updateBg();
} else {
    setBg();
}
```

On utilise ici `getItem()` pour rechercher la valeur liée à la clef « `bgtheme` ». Si une valeur est trouvée, c'est que l'objet de stockage existe déjà et nous n'avons donc pas à le créer. On va alors simplement se contenter de mettre à jour les préférences de l'utilisateur grâce à une fonction `updateBg()`.

La fonction `updateBg()` va récupérer la dernière valeur de l'objet de stockage `bgtheme` et va l'utiliser pour mettre à jour la couleur de fond de la page.

```
function updateBg(){
    let bg = localStorage.getItem('bgtheme');
    htmlElt.style.backgroundColor = '#' +bg;
    bgColor.value = bg;
}
```

On récupère la dernière valeur de l'objet de stockage avec `localStorage.getItem('bgtheme')`. On utilise ensuite cette valeur pour mettre à jour la couleur de fond de notre élément `html`. On en profite également pour modifier la valeur visible du champ du formulaire.

Si aucune valeur n'est trouvée par `getItem()`, cela signifie que l'objet de stockage n'existe pas et il faut le créer. Pour cela, on va créer une fonction `setBg()` qui va utiliser `setItem()` comme ceci :

```
function setBg(){
    localStorage.setItem('bgtheme',bgColor.value);
    updateBg();
}
```

Notre fonction `setBg()` crée un objet de stockage dont la clef est `bgtheme` et la valeur est égale à celle de l'attribut `value` de notre champ de formulaire. Elle exécute également la fonction `updateBg()` qui va servir à mettre à jour la couleur de fond en soi.

Finalement, on va vouloir que ces changements s'effectuent en temps réel. Pour cela, on va utiliser un gestionnaire d'événements pour l'évènement `change`.

```
bgColor.addEventListener('change',setBg);
```

Dès qu'un utilisateur quitte le champ de formulaire, la couleur de fond de la page est mise à jour. Si l'utilisateur quitte le site et revient plus tard, les changements sont enregistrés et la dernière couleur choisie est conservée.

Notez par ailleurs qu'un évènement `StorageEvent` est lancé dès qu'un changement est apporté à un objet de stockage `localStorage`. L'idée principale à retenir ici est que l'évènement est déclenché dans toutes les pages ayant accès à l'objet excepté pour la page courante.

Cela permet aux autres pages d'un domaine qui utilisent le même objet d'appliquer automatiquement les mêmes changements que ceux effectués sur la page courante.

Utiliser l'API de stockage IndexedDB en JavaScript

En plus de Web Storage, il existe une autre API qui va nous permettre de stocker des données côté client : l'API IndexedDB.

Présentation de l'API IndexedDB

L'API IndexedDB est une API de stockage de données côté client qui va être utilisée pour stocker des quantités importantes de données structurées.

La quantité de données qui va pouvoir être stocké est beaucoup plus grande que ce qu'on pourrait stocker avec Web Storage et cela rend donc IndexedDB plus puissante que Web Storage.

IndexedDB est un système de gestion de bases de données transactionnel. On peut le comparer à d'autres systèmes de gestion de base de données basés que le SQL mais, à la différence de ces derniers, IndexedDB est orienté objet.

On va donc pouvoir stocker des objets sous la forme clef / valeur tout comme on a déjà pu le faire avec Web Storage mais, à la différence des données stockées avec Web Storage, on va ici pouvoir stocker plus ou moins n'importe quel type de valeur et définir également différents types de clefs.

Notez par ailleurs que les opérations effectuées par IndexedDB sont réalisées de manière asynchrone, et ceci afin de ne pas bloquer le reste de la page.

Notez également qu'IndexedDB respecte la politique de même origine, ce qui signifie qu'on pourra accéder aux données stockées pour le domaine courant uniquement.

En pratique, pour utiliser IndexedDB, on suivra le schéma suivant :

1. On ouvre une connexion à la base de données
2. On crée un objet de stockage ;
3. On initie une transaction ;
4. On effectue des requêtes ;
5. On crée des gestionnaires d'événements liés au résultat de nos requêtes.

L'objectif de ce chapitre est de mettre en œuvre de ces concepts.

Ouverture de la connexion à la base de données

Pour travailler avec IndexedDB, nous allons avant tout devoir ouvrir une base de données.

Pour cela, on va utiliser la propriété `IndexedDB` qui est une propriété du mixin `WindowOrWorkerGlobalScope` (implémenté par `window`). Cette propriété renvoie un objet `IDBFactory`.

L'interface `IDBFactory` fait partie de l'API IndexedDB et permet aux applications d'accéder à des bases de données de façon asynchrone. Cette interface nous fournit une méthode `open()` qui permet d'ouvrir une connexion à une base de données.

On va donc utiliser cette méthode `open()` avec notre objet (propriété) `IndexedDB`. La méthode `open()` prend en argument obligatoire le nom de la base de données qu'on souhaite ouvrir ainsi que la version de cette base de données en argument facultatif.

```
| let openRequest = indexedDB.open('db',1) ;
```

La méthode `open()` renvoie un objet `IDBOpenRequest` et effectue l'opération d'ouverture de la connexion à la base de données de manière asynchrone.

Si l'ouverture réussit, un évènement `success` est déclenché sur l'objet `IDBOpenRequest` renvoyé par `open()`. La propriété `result` de cet évènement aura alors comme valeur la valeur de l'objet `IDBDatabase` associé à la connexion.

Si l'ouverture de la connexion échoue, un évènement `error` est déclenché sur l'objet `IDBOpenRequest` renvoyé par `open()`.

La version de la base de données détermine son organisation et notamment les objets stockés et leur structure. Par défaut, le numéro de version retenu est 1.

Si le numéro de version de la base de données qu'on souhaite ouvrir est inférieur au numéro fourni à `open()`, un évènement `upgradeneeded` est déclenché pour nous permettre de mettre à jour la base de données. Si la mise à jour se passe bien, un évènement `success` est déclenché.

```
let db='';
let openRequest = indexedDB.open('db',1);

openRequest.onupgradeneeded = function(){
    db = openRequest.result;
    // Operations
}

openRequest.onerror = function(){
    alert('Impossible d\'accéder à IndexedDB');
};

openRequest.onsuccess = function(){
    db = openRequest.result;
    // Operations
}
```

Ici, on crée trois gestionnaires qui vont gérer les évènements `success`, `error` et `upgradeneeded`.

Lorsqu'on crée une nouvelle base de données, ou si on met à jour la version de notre base de données, on doit créer les nouveaux objets de stockage pour cette version de la base dans le gestionnaire de `upgradeneeded`. Les objets créés dans la version précédente seront automatiquement disponibles ; il est inutile de les copier.

De plus, si on essaie de créer un objet de stockage avec un nom déjà existant (ou si on essaie de supprimer un objet de stockage avec un nom qui n'existe pas encore), une erreur sera renvoyée.

Notez que si l'évènement `upgradeneeded` quitte avec succès, l'évènement `success` de la requête d'ouverture de la base de données sera déclenché.

Dans le cas où l'évènement `success` est déclenché (cas où la connexion s'est effectuée avec succès), `openRequest.result` est une instance de `IDBDatabase` et va donc représenter notre connexion.

Création d'un objet de stockage ou « object store »

Les objets de stockage vont stocker les données. Si vous connaissez un petit peu le fonctionnement des bases de données MySQL ou autres, vous pouvez considérer que nos objets de stockage vont être l'équivalent des tables.

Une base de données peut avoir plusieurs objets de stockage et ces objets de stockage peuvent stocker quasiment toutes formes de données. Ces objets de stockage peuvent stocker plusieurs valeurs, et chaque valeur doit être associée à une clef unique au sein d'un objet de stockage.

On va pouvoir passer la clef manuellement en même temps qu'on ajoute une valeur dans l'objet de stockage (ce qui peut être pratique dans le cas où on stocke une valeur primitive) ou définir une propriété qui servira de clef dans le cas où on stocke des objets. On peut également demander à ce que les clefs soient générés automatiquement.

La création ou la modification des objets de stockage va toujours se faire lors de la mise à jour de la version de la base de données, c'est-à-dire au sein du gestionnaire d'évènements `upgradeneeded`.

Pour créer un objet de stockage, on va utiliser la méthode `createObjectStore()`. Cette méthode prend le nom de l'objet de stockage en premier argument ainsi qu'un objet (facultatif) en second argument qui va nous permettre de définir une clef et renvoie un objet appartenant à l'interface `IDBObjectStore`.

Pour définir une clef, on va utiliser l'une des propriétés `keyPath` ou `autoIncrement` de cette interface.

La propriété `keyPath` nous permet de définir une propriété qu'`IndexedDB` utilisera comme clef.

La propriété `autoIncrement` prend une valeur booléenne. Si la valeur passée est `true`, alors la clef pour chaque objet stocké sera générée automatiquement, en s'incrémentant à chaque fois.

```
openRequest.onupgradeneeded = function(){
  db = openRequest.result;
  //If the users storage object does not exist, we create it
  if(!db.objectStoreNames.contains('users')){
    db.createObjectStore('users', {keyPath: 'id'});
  }
}
```

Initiation d'une transaction

On appelle « transaction » un groupe d'opérations dont le destin est lié. L'idée principale à retenir à propos des transactions est la suivante : les différentes opérations doivent toutes réussir indépendamment pour que la transaction soit un succès. Si une opération échoue, alors la transaction et donc l'ensemble des opérations échouent.

Dans notre contexte, les transactions vont s'effectuer à partir de l'objet symbolisant la connexion à la base de données (notre instance de `IDBDatabase`). Pour démarrer une nouvelle transaction, nous allons utiliser la méthode `transaction()` à partir de cet objet.

Cette méthode va prendre deux arguments : la liste d'objets de stockage que la transaction va traiter (obligatoire) ainsi que le type ou mode de transaction souhaité (facultatif).

On peut choisir parmi trois modes de transaction : `readonly` (lecture seule), `readwrite` (lecture et écriture) et `versionchange` (changement de version). Ces modes vont définir quelles manipulations on va pouvoir effectuer sur les données. Par défaut, le mode est `readonly`.

Pour lire les enregistrements d'un objet de stockage existant, la transaction peut être en mode `readonly` ou `readwrite`. Pour appliquer des changements à un objet de stockage existant, la transaction doit être en mode `readwrite`.

Pour changer la structure de la base de données (le schéma), ce qui implique de créer ou supprimer des objets de stockage ou des index, la transaction doit être en mode `versionchange`.

Création de requêtes et gestion des résultats

IndexedDB nous permet d'ajouter, de supprimer, de récupérer ou de mettre à jour des données dans notre base de données.

En pratique, pour effectuer ces manipulations, on commencera par créer une transaction puis on récupérera l'objet de stockage de celle-ci.

Ensuite, on va effectuer des requêtes (ajout de données, suppression, etc.) à partir de cet objet `IDBObjectStore` et on va finalement gérer les cas de succès ou d'erreur liés au résultat de nos requêtes.

L'interface `IDBObjectStore` nous fournit les différentes méthodes qui vont nous permettre de manipuler nos objets de stockage et notamment :

- Les méthodes `put()` et `add()` pour stocker des données dans la base ;
- Les méthodes `get()` et `getAll()` pour récupérer les données depuis la base ;
- Les méthodes `delete()` et `clear()` pour supprimer des données.

Pour stocker une nouvelle valeur dans un objet de stockage, par exemple, on pourra écrire un script comme celui-ci :

```
let db = '';
let openRequest = indexedDB.open('db', 1);

openRequest.onupgradeneeded = function(){
  db = openRequest.result;

  //Si l'objet de stockage users n'existe pas, on le crée
  if (!db.objectStoreNames.contains('users')){
    db.createObjectStore('users', {keyPath: 'id'});
  }
};

openRequest.onerror = function(){
  alert('Impossible d\'accéder à IndexedDB');
};

openRequest.onsuccess = function(){
  db = openRequest.result;
  let transaction = db.transaction('users', 'readwrite');

  transaction.oncomplete = function(){
```

```

    alert('Transaction terminée');
  };

  let users = transaction.objectStore('users');

  let user = {
    id: 1,
    prenom: 'Pierre-Antoine',
    mail: 'pierre-antoine.legrand@epul3a.local',
    inscription: new Date()
  };

  let ajout = users.add(user);

  ajout.onsuccess = function(){
    alert('Utilisateur ajouté avec la clef ' + ajout.result);
  };

  ajout.onerror = function(){
    alert('Erreur : ' + ajout.error);
  };
};

```

Ici, on commence donc par initier une transaction à partir de notre objet représentant la connexion à notre base de données (objet appartenant à `IDBDatabase`).

Notre objet `let transaction` appartient à `IDBTransaction`. Cette interface possède une méthode `objectStore()` qui renvoie un objet `IDBObjectStore`.

La ligne `transaction.objectStore()` nous permet donc d'accéder à notre objet de stockage afin d'effectuer des opérations avec celui-ci. On place le résultat dans une variable qui est un objet `IDBObjectStore`.

Ici, on utilise la méthode `add()` de l'interface `IDBObjectStore` qui permet de stocker de nouvelles valeurs dans un objet de stockage. Cette méthode prend une valeur en argument obligatoire et une clef en argument facultatif (la clef est fournie automatiquement seulement si l'objet de stockage ne possède pas d'option `keypath` ou `autoIncrement`).

Pour information, la différence entre les méthodes `put()` et `add()` est la suivante : si on fournit une clef qui existe déjà pour une valeur à `put()`, la clef sera modifiée tandis qu'avec `add()` la requête échouera et une erreur sera générée.

On effectue donc ici la requête suivante : « ajoute une nouvelle valeur dans notre objet de stockage ». Nous n'avons alors plus qu'à mettre en place les gestionnaires d'évènements de succès et d'erreur pour cette requête.

Notre objet `let request` appartient ici à l'interface `IDBRequest`. Cette interface dispose d'une propriété `result` qui contient le résultat d'une requête.

Lorsqu'on l'utilise avec une requête de type `add()`, la valeur de `request.result` est la clef de la valeur qui vient d'être ajoutée.

Cette interface contient également une propriété `error` qui indique le code de l'erreur survenue durant le traitement de la requête.

On va également pouvoir de manière similaire récupérer des données dans la base ou en supprimer. Pour récupérer une donnée en particulier, on pourra par exemple utiliser la méthode `get()`. Cette méthode prend la clef de la valeur qu'on souhaite récupérer en argument.

```
let db = '';
let openRequest = indexedDB.open('db', 1);

openRequest.onupgradeneeded = function(){
    db = openRequest.result;

    //Si l'objet de stockage users n'existe pas, on le crée
    if (!db.objectStoreNames.contains('users')){
        db.createObjectStore('users', {keyPath: 'id'});
    }
};

openRequest.onerror = function(){
    alert('Impossible d\'accéder à IndexedDB');
};

openRequest.onsuccess = function(){
    db = openRequest.result;
    let transaction = db.transaction('users', 'readwrite');

    transaction.oncomplete = function(){
        alert('Transaction terminée');
    };

    let users = transaction.objectStore('users');

    let user = {
        id: 1,
        prenom: 'Pierre-Antoine',
        mail: 'pierre-antoine.legrand.giraud@epul3a.local',
        inscription: new Date()
    };

    let ajout = users.put(user);

    ajout.onsuccess = function(){
        alert('Utilisateur ajouté avec la clef ' + ajout.result);
    };

    ajout.onerror = function(){
        alert('Erreur : ' + ajout.error);
    };

    let lire = users.get(1);
    lire.onsuccess = function(){
        alert('Nom de l\'utilisateur 1 : ' + lire.result.prenom);
    };

    lire.onerror = function(){
        alert('Erreur : ' + lire.error);
    };
};
```

```
};  
};
```

On va également pouvoir supprimer des données en utilisant par exemple la méthode `delete()` pour supprimer une ou plusieurs données choisies. Cette méthode prend la clef liée à la valeur qu'on souhaite supprimer en argument ou un objet représentant un intervalle de clefs liées aux valeurs qu'on souhaite supprimer.

```
let db = '';  
let openRequest = indexedDB.open('db', 1);  
  
openRequest.onupgradeneeded = function(){  
    db = openRequest.result;  
  
    //Si l'objet de stockage users n'existe pas, on le crée  
    if (!db.objectStoreNames.contains('users')){  
        db.createObjectStore('users', {keyPath: 'id'});  
    }  
};  
  
openRequest.onerror = function(){  
    alert('Impossible d\'accéder à IndexedDB');  
};  
  
openRequest.onsuccess = function(){  
    db = openRequest.result;  
    let transaction = db.transaction('users', 'readwrite');  
    transaction.oncomplete = function(){alert('Transaction terminée')};  
    let users = transaction.objectStore('users');  
  
    let user = {  
        id: 1,  
        prenom: 'Pierre-Antoine',  
        mail: 'pierre-antoine.legrand.giraud@epul3a.local',  
        inscription: new Date()  
    };  
  
    let ajouter = users.put(user);  
    ajouter.onsuccess = function(){  
        alert('Utilisateur ajouté avec la clef ' + ajouter.result);  
    };  
    ajouter.onerror = function(){  
        alert('Erreur : ' + ajouter.error);  
    };  
  
    let lire = users.get(1);  
    lire.onsuccess = function(){  
        alert('Nom de l\'utilisateur 1 : ' + lire.result.prenom);  
    };  
    lire.onerror = function(){  
        alert('Erreur : ' + lire.error);  
    };  
};
```

```
let supprimer = users.delete(1);
supprimer.onsuccess = function(){
    alert('Utilisateur supprimé de la base de données');
};
supprimer.onerror = function(){
    alert('Erreur : ' + supprimer.error);
};
};
```

En résumé

L'API IndexedDb permet de stocker des quantités importantes de données structurées dans le navigateur de vos visiteurs.

Ces API fonctionne principalement de manière asynchrone et adhère au principe de « same-origin policy » (politique de même origine).

IndexedDB est une API orienté objet : les données vont être stockées dans des objets de stockage ou « object store ». Les données sont stockées sous la forme de paires clef / valeur. Les valeurs peuvent être des objets structurés, et les clés peuvent être des propriétés de ces objets.

Cette API est construite autour d'un modèle de base de données transactionnelles : les différentes manipulations vont s'effectuer dans un contexte de transaction.

Durant ces transactions, on va effectuer des requêtes pour manipuler nos données. Ces requêtes sont des objets qui reçoivent les événements DOM de succès ou d'échec.